



Degree Project in Computer Science and Engineering

Second cycle, 30 credits

Lightweight Memory Networks for Interaction Prediction

Generalization of the LiMNet Architecture

TITOUAN MAZIER

Lightweight Memory Networks for Interaction Prediction

Generalization of the LiMNet Architecture

TITOUAN MAZIER

Master's Program, Computer Science

Date: August 11, 2025

Supervisors: Šarūnas Girdzijauskas and Lodovico Giaretta

Examiner: Viktoria Fodor

School of Electrical Engineering and Computer Science

Host company: RISE, Research Institute of Sweden

Swedish title: Lightweight Memory Networks för interaktion prediktion

Swedish subtitle: Generalization av LiMNet Arkitektur

Abstract

User-item recommendation is a central challenge for search engines, social media platforms, and streaming services, due to the need to model both relational structures and temporal dynamics. Many existing solutions address these two aspects separately, limiting their ability to fully capture user behavior.

In this work, we attempt to bridge that gap by evaluating Lightweight Memory Networks (LiMNet), a model designed to preserve causal relationships within sequences of temporal interactions. To assess its potential, we developed a benchmarking framework for user-item interaction prediction. We compared LiMNet against Jodie, a state-of-the-art baseline, across three real-world datasets: Wikipedia page edits, Reddit post submissions, and LastFM music streams. These datasets vary in scale and interaction patterns, providing a comprehensive testbed.

Our results show that while LiMNet offers advantages in efficiency and adaptability, it consistently underperforms compared to Jodie in predictive accuracy. Additionally, our findings hint at a consistent bias across all datasets toward short-term global popularity. This suggests that existing models may be overfitting to recent trends rather than learning long-term user preferences, highlighting a potential limitation in the current evaluation paradigms.

Keywords

Graph Representation Learning, Temporal Interaction Networks, Interaction Prediction, Link Prediction, Data Mining, Machine Learning, Recommendations

Sammanfattning

User-item rekommendation är en central utmaning för sökmotorer, sociala medieplattformar och streamingtjänster, eftersom det kräver att både relationella strukturer och temporala dynamiker modelleras samtidigt. Många befintliga lösningar hanterar dessa två aspekter separat, vilket begränsar deras förmåga att fullt ut fånga användarbeteenden.

I detta arbete försöker vi överbrygga detta gap genom att utvärdera Lightweight Memory Networks (LiMNet), en modell som är utformad för att bevara kausala samband i sekvenser av temporala interaktioner. För att undersöka modellens potential utvecklade vi ett benchmark-ramverk för user-item interaktionsprediktion. Vi jämförde LiMNet med Jodie, en avancerad referensmodell, över tre verkliga datasets: Wikipedia-redigeringar, Reddit-inlägg och musikströmningar från LastFM. Dessa datasets varierar i skala och interaktionsmönster, vilket ger en bred testbas.

Våra resultat visar att även om LiMNet erbjuder fördelar vad gäller effektivitet och anpassningsbarhet, så presterar den konsekvent sämre än Jodie i prediktiv noggrannhet. Dessutom antyder våra fynd att det finns en genomgående bias i alla datasets mot kortsiktig global popularitet. Detta tyder på att befintliga modeller tenderar att överanpassa till nyliga trender snarare än att lära sig användares långsiktiga preferenser, vilket pekar på en potentiell begränsning i nuvarande utvärderingsmetoder.

Nyckelord

Graph Representation Learning, Temporal Interaction Networks, Link Prediction, Data Mining, Machine Learning, Rekommendation

Résumé

La recommandation d'information représente un défi majeur pour les moteurs de recherche, les plateformes de réseaux sociaux et les services de streaming, car elle nécessite de modéliser à la fois les structures relationnelles et les dynamiques temporelles. De nombreuses solutions existantes traitent ces deux aspects séparément, ce qui limite leur capacité à capturer pleinement le comportement des utilisateurs.

Dans ce travail, nous cherchons à combler cette lacune en évaluant Lightweight Memory Networks (LiMNet), un modèle conçu pour préserver les relations causales dans les séquences d'interactions temporelles. Pour évaluer son potentiel, nous avons développé un framework pour l'évaluation de prédiction d'interaction. Nous avons comparé LiMNet à Jodie, un modèle de référence, sur trois jeux de données réels : des modifications de pages Wikipédia, des publications Reddit et des streams de musique sur LastFM. Ces jeux de données diffèrent en termes d'échelle et de types d'interaction, offrant ainsi un support varié pour tester les modèles.

Nos résultats montrent que, bien que LiMNet présente des avantages en matière d'efficacité et de flexibilité, il affiche systématiquement des performances inférieures à celles de Jodie en termes de précision prédictive. Par ailleurs, nos observations suggèrent un biais constant dans tous les jeux de données en faveur des tendances globales à court terme. Cela suggère que les comportements rassemblés dans les jeux de données utilisés pourraient s'expliquer majoritairement par des phénomènes de courte durée, au détriment de phénomènes plus profonds causés par les préférences des utilisateurs à long terme — une limite potentielle des paradigmes d'évaluation actuels.

Keywords

Apprentissage de représentation de graphes., Réseaux d'interaction temporels, Prédiction de liens, Data Mining, Machine Learning, Recommendations

Acknowledgments

I would like to express my sincere gratitude to my supervisors, Lodovico Giaretta and Šarūnas Girdzijauskas, for their continuous guidance, support, and encouragement throughout the course of this thesis. Their expertise and feedback were invaluable at every stage of this work.

I would also like to thank RISE - Research Institutes of Sweden for providing me with access to their facilities, which offered a productive environment to carry out this research.

Finally, I gratefully acknowledge BSC - Barcelona Supercomputing Center for granting access to their supercomputing infrastructure, which played a crucial role in enabling the large-scale experiments presented in this thesis.

Stockholm, August 2025

Titouan Mazier

Contents

1. Introduction	1
1.1. Motivation	1
1.2. Problem	2
1.3. Limitations	3
1.4. Contributions	3
2. Background	5
2.1. User-Item Link Prediction	5
2.2. Graph Representation Learning	6
2.3. Dynamic Graphs	7
2.4. Cross-RNN	8
2.5. LiMNet	10
2.6. Jodie	10
3. Method	13
3.1. Datasets	13
3.2. Experimental framework	14
3.2.1. Data preparation	15
3.2.2. Batching Strategy	15
3.2.3. Evaluation and Training loop	16
3.2.4. Embedding comparison	17
3.2.5. Code Reusability	18
3.3. Adaptations of the Lightweight Memory Networks (LiMNet) architecture	19
3.3.1. Loss functions	19
3.3.2. Addition of time features	20
3.3.3. Embedding normalization	21
3.3.4. Stacking of multiple LiMNet layers	21
3.4. Baseline	22
4. Experiments	23
4.1. Improvements on LiMNet	23
4.1.1. Adding time features	23
4.1.2. Normalizing the embeddings	24
4.1.3. Stacking layers	25
4.2. Impact of the sequence size on the results	26
5. Conclusions	29
5.1. Limitations	30

5.2. Future Works	30
5.3. Ethics and Sustainability	31
5.3.1. Ethical aspects	31
5.3.2. Societal impact	31
5.3.3. Sustainability	31
References	33
A. Complexity	37

List of Figures

Figure 2.1	Architecture of LiMNet.	10
Figure 2.2	Architecture of Jodie.	11
Figure 3.1	Schema of the evaluation framework. Blue indicates that the implementation is tight to the model evaluated.	16
Figure 3.2	Example of two items with significantly distinct daily patterns within the LastFM Dataset.	20
Figure 3.3	Architecture of LiMNet with two layers.	21
Figure 4.1	Comparison of LiMNet and Jodie models.	23
Figure 4.2	Performances of the LiMNet model with time features added.	24
Figure 4.3	Performances of the LiMNet model with and without normalization of the embeddings.	25
Figure 4.4	Performances of the LiMNet model with various numbers of stacked layers.	26
Figure 4.5	Effect of the sequence length on the models' performances. . .	28

List of Tables

Table 3.1	Details of the datasets.	14
Table 1.1	Symbols used for the complexity analysis.	37
Table 1.2	Summary of the memory complexity for LiMNet and Jodie . .	38

Acronyms and Abbreviations

CTDG – Continuous-Time Dynamic Graph: Dynamic graph where each edge is associated with a timestamp. 7, 7

GRL – Graph Representation Learning: The task of computing high level representation of graphs, subgraphs, nodes or edges. Used as inputs for Machine Learning algorithms. 6, 6, 7, 7

GRU – Gated Recurrent Unit: A type of recurrent neural network that simplifies the LSTM architecture by combining the forget and input gates into a single update gate. 9, 9, 9, 9

LiMNet – Lightweight Memory Networks: A neural network architecture that focuses on efficient memory utilization and lightweight design for temporal interaction network embeddings computations. ix, ix, ix, xi, xiii, 2, 2, 2, 2, 3, 3, 3, 3, 4, 4, 4, 10, 10, 10, 10, 10, 10, 11, 11, 12, 13, 15, 19, 19, 19, 19, 20, 21, 21, 21, 21, 22, 22, 23, 23, 23, 23, 23, 23, 24, 24, 25, 25, 26, 29, 29, 29, 29, 29, 29, 30, 30, 30, 30, 31, 31, 31, 31, 37, 37, 37, 38, 38, 38

LSTM – Long-Short Term Memory: A type of recurrent neural network designed to effectively learn and remember short and long-term dependencies in sequential data by using specialized memory cells and gating mechanisms. 9, 9, 9, 9

MRR – Mean Reciprocal Rank: Metric used to compute the success of a ranking algorithm. It is a value comprised between 0 and 1 where 0 indicates a total absence of the expected result in the ranked items and 1 means that the expected item is always ranked first. 23, 24, 27

RNN – Recurrent Neural Network: A neural network whose output depends on both the inputs and on the state of an internal memory that is updated with each input. 8, 8, 8, 9, 9, 9, 9

Chapter 1

Introduction

1.1. Motivation

The rapid expansion of digital technologies has led to an overwhelming abundance of information, making it increasingly difficult to identify relevant and meaningful content [1], [2]. In response to this challenge, search engines and recommendation systems have become essential tools for filtering and navigating vast data landscapes [3].

Both systems share a common objective: to deliver information that is most relevant to the user. Rather than applying a one-size-fits-all filter, content personalization techniques have emerged as a particularly effective approach that, instead, adapt their output based on user-specific context such as search history, demographics, and past interactions [4]. While content personalization is a natural solution for recommendation systems, it is also highly effective in enhancing search engine performance [5]. For instance, a search query for the term “football” should yield different results for a user interested in American football compared to someone seeking information about association football (soccer).

Content personalization can be represented as an algorithm that takes user-specific information as input and produces a ranked list of items from a larger catalog. These items can include any kind of content available to the user, such as songs in a music streaming service or web pages returned by a search engine.

However, evaluating the performance of such algorithms typically requires knowledge of which items are genuinely relevant to each user, knowledge that can be costly and, in some cases, impossible to collect. Therefore, user behaviors such as clicks, listens, or edits, that are easy to collect at scale are often used instead [6]. As a result, content recommendation is often approximated as an interaction prediction task, where the goal is to predict future user-item interactions based on past behavior [7].

Interaction prediction is a self-supervised learning task in which past user-item interactions serve first as labels and later as input features used to

predict future interactions. Unlike other self-supervised tasks, it is distinguished by the inherently relational nature of the data: each interaction connects users and items, forming a web of dependencies that influence future behavior. As a result, these interactions are often modeled as user-item networks to highlight the structural relationships between entities[8].

Besides, the temporal dimension, which includes the order and timing of interactions, typically plays a critical role in accurately modeling and understanding user behavior over time[9]. One natural way to model both temporal and relational information is to use a temporal interaction network, that is, a network where each interaction is linked to a timestamp. This way, following the order of the timestamps reveals the network and its evolutions, presenting an ever-changing map of relationships between the users and the items and allowing for more nuanced and dynamic predictions. Yet, few solutions rely on this model, despite its intuitive definition, which motivated this work to look further into existing solutions[10].

LiMNet [11] is a simple machine learning model designed to process temporal interactions in a causal manner, leveraging both relational information and the order of interactions. The model has demonstrated strong performance in tasks such as botnet detection in IoT networks, and it is built in a modular, adaptive way that makes it easy to apply to other problems. Given these promising characteristics and its ability to exploit precisely the types of information that make interaction prediction challenging, LiMNet appears to be a compelling candidate for this task.

1.2. Problem

The core research question guiding this work is:

“Can an adaptation of LiMNet have the potential to compete against state-of-the-art models on the task of interaction prediction for user-item network?”

LiMNet has shown promising results in specialized domains such as IoT botnet detection [11] and cryptocurrency fraud detection [12], but its capabilities have not yet been tested in the context of user-item interaction prediction. This gap raises important questions about the generalizability of LiMNet and whether its lightweight, causal and scalable architecture can match the performance of more established, domain-specific models. Interaction prediction also differs fundamentally from botnet or fraud detection

tasks, as it is a ranking task rather than a classification one, necessitating additional adaptations to the LiMNet model.

In addressing this central question, the thesis pursues two main objectives:

- Evaluate the performance of the original LiMNet architecture on user-item interaction prediction tasks.
- Explore the architectural potential of LiMNet by implementing and evaluating a series of targeted improvements.

To this end, we develop a flexible evaluation framework supporting multiple models while ensuring fair and consistent comparisons. Additionally, we implement and evaluate several architectural adaptations to LiMNet, aiming not only to improve its performance on interaction prediction but also to further investigate the design space enabled by its modular structure.

1.3. Limitations

One alternative approach to interaction prediction involves building a classifier to determine whether a given user is likely to interact with a specific item[13]. This approach is excluded from the present work, which instead focuses on computing embeddings and generating ranked item lists.

Another limitation of this study is its strict focus on user-item networks. Interactions between entities of the same type, such as user-user or item-item connections, are not considered. For instance, tasks like friendship prediction on a social network fall outside the scope of this project. Consequently, each entity is strictly classified as either a user or an item, with no allowance for hybrid or hierarchical roles. And only one kind of interaction is considered, e.g. user listening to a song but not artist posting a song.

Furthermore, this work considers only punctual interactions, i.e. discrete events occurring at specific points in time. Continuous interactions are excluded, and due to data limitations, the duration of interactions is not modeled. Although interaction durations could offer meaningful insights into engagement or relevance, they remain outside the scope of this thesis.

1.4. Contributions

The main contributions of this project are as follows:

- Development and publication of a framework for evaluating models on the user-item interaction prediction task.

- Presentation and evaluation of LiMNet, an embedding model originally proposed for IoT botnet detection.
- Proposal and testing of architectural modifications to LiMNet aimed at improving its performance in the context of user-item interaction prediction.
- Reproduction and evaluation of Jodie, a state-of-the-art model for temporal interaction prediction.

Interestingly, our implementation produced results that exceeded expectations.

- Identification of a possible structural bias in the benchmark datasets, suggesting they may favor global popularity trends over more complex, long-term behavioral patterns.

Our evaluation demonstrates that, in its simplified form, LiMNet does not achieve competitive performance with state-of-the-art baseline and is therefore not well-suited for the task of interaction prediction. However, we identified a meaningful improvement by normalizing the embeddings stored in the model's memory, which significantly enhanced predictive accuracy, even though not enough to match the baseline. Additionally, we observed that varying the sequence length had minimal impact on performance, suggesting that the datasets are dominated by short-term global trends rather than long-term user-specific behaviors.

Chapter 2

Background

This chapter provides the background for the project. In Section 2.1, we provide an overview of the more standard link prediction task and the classical solutions for the problem. In Section 2.2, we further develop the concept of graph embedding and some common methods to create them. Then, in Section 2.3, we discuss the addition of a time dimension in graph-shaped data and the way it can be exploited, followed by a presentation of cross-RNN architectures in Section 2.4. Finally, we present the model of interest for this work in Section 2.5 and why we believe that it is a relevant addition to the task of interaction prediction.

2.1. User-Item Link Prediction

Interaction prediction can often be formulated as a link prediction problem within a user-item graph. In such a graph, each user and each item is represented as a node. Each user interaction with an item is registered as an edge in the graph. Predicting future interactions comes down to predicting which edges are likely to appear next.

In this project, we focus specifically on user-item interaction networks—systems in which all interactions occur between distinct user and item entities. Users and items play fundamentally different roles: users initiate interactions, while items are the targets. This introduces a structural asymmetry in the network, where interactions are always directed from a user to an item. In practice, this means that prediction tasks are framed from the user’s perspective, with the goal of identifying the most relevant items they are likely to interact with next.

There are two primary approaches to this problem: one based on graph analytics and the other on feature-based methods[14]. Graph analytics focuses on measuring the proximity between a user and various items in the graph, leveraging insights from the user’s past interactions and the behaviors of similar users. For instance, if two users have listened to the same set of songs, one may likely enjoy the songs the other has listened to. Graph theory offers

a variety of methods to compute closeness between nodes, including shortest path lengths, the number of shared neighbors, or the exclusivity of those shared neighbors.

The second approach leverages additional information beyond the user-item relationship. Most real-world systems provide rich metadata about interactions, users, and items. For example, in a music streaming service, a song may include attributes like genre and duration, while a user may be characterized by age or selected language. These attributes are referred to as features, and utilizing them is central to machine learning approaches. Unlike graph-based methods, feature-based models recommend items by identifying similar users based on shared characteristics. Continuing the music example, the system might learn that songs with lyrics in Swedish are less likely to appeal to users that don't use Swedish as their primary language.

The challenge lies in integrating both approaches. Lichtenwalter et al. proposed framing link prediction as a supervised machine learning task, where the objective is to predict whether an edge will form between a given user-item pair in the future. To incorporate relational data into the model, graph closeness metrics are added to the user and item features[15]. This setup uses graph structure not as the direct basis for prediction but as a source of enriched features, enabling machine learning algorithms to work with abstracted representations of the graph.

Despite their strengths, these methods share a common drawback: for each user, a score must be computed for every possible item to generate a recommendation[16]. This becomes computationally infeasible when dealing with large item catalogs. A common solution is to learn high-dimensional embeddings for users and items separately and then compute a similarity score, such as a dot product or distance measure, to rank items[17]. This transforms the problem into a nearest-neighbor search, a well-studied task with many efficient and scalable solutions.

2.2. Graph Representation Learning

The task of learning high-level representations from graph data is known as Graph Representation Learning (GRL). GRL encompasses a broad family of machine learning techniques aimed at transforming the complex, non-Euclidean structure of graphs into low-dimensional Euclidean representations

(i.e., numerical vectors), making them suitable for use in downstream machine learning tasks.

GRL methods can be applied to a wide range of problems. These include classifying entire graph structures, such as molecular graphs; extracting sub-graph representations from knowledge graphs to be used in large language models; and, most commonly, generating node embeddings. These node embeddings must encode not only the intrinsic features of individual nodes but also the context in which they appear. This context typically includes neighboring nodes and their corresponding features and positions within the graph.

2.3. Dynamic Graphs

Much of the information generated in real-world networks is inherently dynamic, particularly in user-item interaction networks, where each interaction occurs at a specific point in time. Despite this, the temporal dimension is often ignored or simplified in order to reduce modeling complexity. Yet, temporal information carries unique value, enabling models to capture not just patterns but also the causal relationships between interactions.

Causality refers to the principle that actions can influence future outcomes. This is especially important when studying processes that propagate through networks, such as the spread of information, trends, or behaviors. In these scenarios, an interaction may change the state of the involved nodes, making it necessary to treat the same node differently depending on the time of observation. While humans intuitively understand these evolving patterns, many standard GRL techniques disregard temporal order, propagating information across the graph without regard to when interactions occurred.

In their review of dynamic networks [10], Zheng et al. outline two common approaches to incorporating temporal information into two models. The first involves representing the graph as a sequence of discrete snapshots, each corresponding to a specific time step. The second, known as Continuous-Time Dynamic Graph (CTDG), treats each graph update as an event timestamped in continuous time, typically the addition or removal of an edge.

This work focuses on CTDG, where all events are modeled as punctual interactions, also referred to as temporal interaction networks. These networks

offer a faithful representation of many real-world systems, as they continuously track changes over time. However, they come with a challenge: the underlying graph structure becomes ephemeral, with each edge appearing only momentarily. As a result, these networks are often better understood as interaction streams rather than static or evolving graphs with persistent edges.

A popular approach for leveraging temporal data when generating node embeddings is to maintain a memory of embeddings and update them as interactions occur. One foundational model in this domain is DeepCoevolve [18], a model for link prediction that uses two components: a cross-RNN (further detailed in Section 2.4) to update user and item representations, and an intensity function to predict the likelihood of future interactions at any given time. Following DeepCoevolve, several cross-RNN-based models have been proposed, achieving notable performance improvements.

Jodie [19] extends DeepCoevolve by introducing a static embedding component alongside the dynamic one. The cross-RNN tracks the trajectory of users and items over time, while a neural projection layer predicts their future embeddings at different time steps, replacing the intensity function used in DeepCoevolve.

DeePRed [20] builds upon DeepCoevolve with the goal of simplifying and accelerating training by removing recurrence from the cross-RNN mechanism. Instead, dynamic embeddings are computed directly from static embeddings, avoiding recursive updates. The absence of long-term memory is addressed through the use of a sliding context window and an attention mechanism that identifies and weights the most relevant past interactions.

2.4. Cross-RNN

The key mechanism underlying the models discussed in the previous section (DeepCoevolve, Jodie, DeePRed) is known as cross-RNN, where RNN stands for Recurrent Neural Network. A RNN is a type of neural network designed to process sequential data by maintaining and updating a memory state across time steps. Formally, a RNN layer is defined as:

$$o(i_t) = f(i_t, h_{t-1}) \quad (1)$$

$$h_t = g(i_t, h_{t-1}) \quad (2)$$

Here, t denotes the time step of the input i_t . The function $o(i_t)$ produces the layer's output, and h_t represents the updated memory after processing i_t . The functions f and g are parameterized transformations, typically involving learned weights. Popular RNN architectures, such as Long-Short Term Memory (LSTM) and Gated Recurrent Unit (GRU), are designed to retain long-term dependencies more effectively than simple RNNs. LSTM maintains two types of memory (short-term and long-term), whereas GRU simplifies this structure by using a single memory vector with a gating mechanism. In practice, GRUs achieve performance comparable to LSTMs while being computationally less demanding [21].

A cross-RNN layer is another type of neural network layer that extends the concept of a RNN by maintaining separate memory states for all nodes in a graph. At time t , the memory is represented as: $\mathbf{H}_t = (\mathbf{h}_t^u)_{u \in \mathbb{U}} \cup (\mathbf{h}_t^i)_{i \in \mathbb{I}}$ where $\mathbb{U}$ and $\mathbb{I}$ denote the sets of users and items, respectively. For each interaction $(u, i, t, \mathbf{f})$, the memory states of the involved user u and item i are updated according to:

$$\mathbf{h}_t^u = g^{\mathbb{U}}(\mathbf{h}_{t-1}^u, \mathbf{h}_{t-1}^i, t, \mathbf{f}) \quad (3)$$

$$\mathbf{h}_t^i = g^{\mathbb{I}}(\mathbf{h}_{t-1}^i, \mathbf{h}_{t-1}^u, t, \mathbf{f}) \quad (4)$$

For all other nodes $v \in (\mathbb{U} \setminus \{u\}) \cup (\mathbb{I} \setminus \{i\})$, the memory remains unchanged.

Here, $g^{\mathbb{U}}$ and $g^{\mathbb{I}}$ are node-type-specific update functions analogous to g in a standard RNN. LSTM and GRU cells can be used within the cross-RNN framework, with the key distinction that memory management is handled externally to the cell.

The primary advantage of cross-RNN architectures is their inherent preservation of causality. Since updates depend strictly on past states, the model respects the temporal order of interactions by design. However, this sequential nature introduces a limitation: cross-RNN models cannot be parallelized over the sequence during training. Fortunately, this constraint primarily affects training efficiency, during inference, each interaction can be processed independently, making the approach practical for real-time applications.

2.5. LiMNet

LiMNet is a cross-RNN model designed to optimize memory utilization and computational efficiency during inference. In its original formulation [11], LiMNet is part of a comprehensive framework for botnet detection in IoT networks. The framework includes four main components: an input feature map, a generalization layer, an output feature map, and a response layer. For the purposes of this work, however, we consider only the generalization layer. Since the other components are task-specific, the term “LiMNet” in this thesis refers exclusively to the generalization layer.

As a graph embedding module, LiMNet provides a straightforward implementation of a cross-RNN mechanism, Figure 2.1 proposes a visualization of its architecture. This simplicity offers two main advantages. First, LiMNet is highly efficient at inference time, with memory requirements that scale linearly with the number of nodes in the network (a formal complexity analysis is provided in Appendix A). Second, it offers strong flexibility in handling dynamic node sets: when a new node is introduced, its embedding can be computed immediately without retraining the model. Deleting a node is even simpler: its embedding can just be removed from memory without consequences for the other embeddings.

LiMNet has already shown promising results in domains such as IoT botnet detection[11] and cryptocurrency fraud detection[12]. However, it has yet to be evaluated for interaction prediction on user-item networks. Given its design and prior successes, it stands as a compelling candidate for this task.

2.6. Jodie

Jodie, introduced in[19], is a state-of-the-art architecture that, like LiMNet, builds on cross-RNN embeddings. The difference lies in additional output modules added in the Jodie architecture to refine predictions. First, Jodie enriches its dynamic embeddings by concatenating one-hot representations of users and items to form the final embedding vectors. Second, it incorporates the elapsed time between consecutive user interactions through a projection mechanism designed to anticipate the future trajectory of embeddings. Third, it employs a specialized loss function to prevent user and item embeddings from shifting excessively in response to a single interaction.

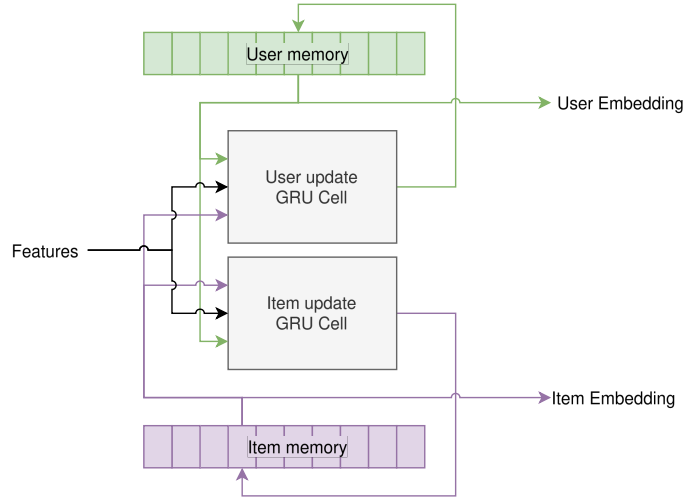


Figure 2.1: Architecture of LiMNet.

Our implementation of Jodie differs from the original in two key ways. We replaced the t-batch algorithm used for creating training batches with fixed-length interaction sequences and omitted interaction features for simplicity.

These differences make Jodie a strong and informative baseline for assessing LiMNet’s ability to generalize to user-item interaction prediction tasks. Moreover, given Jodie’s already strong performance on temporal interaction networks, we hypothesize that removing its additional output modules, down to the LiMNet architecture, could yield notable gains in inference speed and latency as suggested by the theoretical complexity analysis

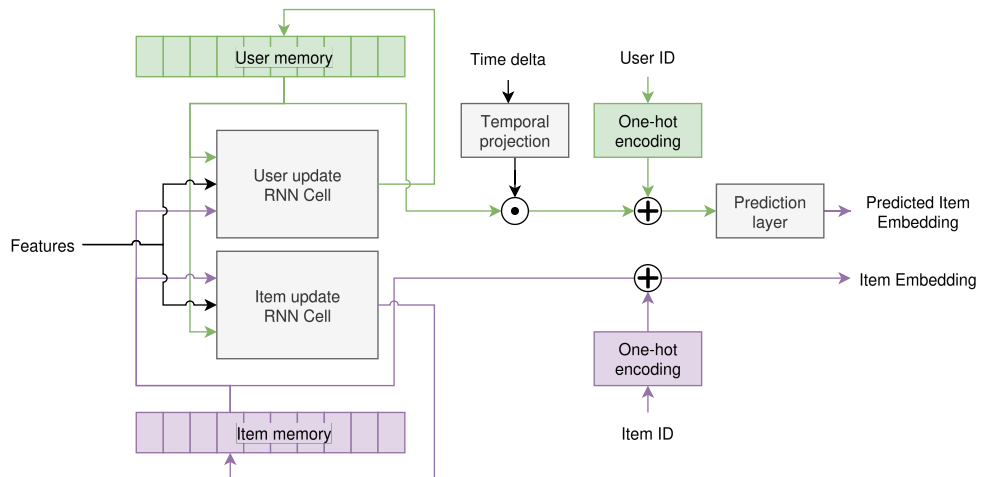


Figure 2.2: Architecture of Jodie.

detailed in Appendix A. Additionally, while Jodie does not require retraining to integrate new interactions, it lacks LiMNet’s ability to dynamically handle the insertion or deletion of users and items.

Chapter 3

Method

In this chapter, we detail the experiments conducted throughout this work. First in Section 3.1, we introduce the datasets used in this work. Section 3.2 details the framework developed to conduct the experiments in a fair and controlled environment. Next, we present in Section 3.3 the various adaptations proposed for LiMNet to solve the task of link-prediction. Finally, we discuss in Section 3.4 exploration we conducted with the baselines.

3.1. Datasets

This project uses three publicly available datasets sourced from the Stanford Large Network Dataset Collection¹. These datasets were originally compiled by Kumar et al. [19] and have since become widely adopted as de facto standard benchmarks for evaluating interaction prediction models.

- **Wikipedia edits:** This dataset captures edits made to Wikipedia pages over the course of one month. It includes the 1,000 most edited pages during that period and 8,227 users who each made at least five edits to these pages. In total, it contains 157,474 interactions.
- **Reddit posts:** Built using a similar methodology as the Wikipedia dataset, this dataset records 672,447 posts made by the 10,000 most active users on the 1,000 most active subreddits within a month.
- **LastFM songs listens:** This dataset logs 1,293,103 music streams performed by 1,000 users on the 1,000 most listened-to songs on the LastFM platform, again over the span of one month.

The original dataset publication also included a fourth dataset containing interactions between 7,047 students and 97 courses on a MOOC platform. However, we excluded this dataset from our experiments, as it does not reflect a relevant use case for interaction prediction. Users on MOOC platforms typically have a clear intent when accessing the platform, which diminishes the predictive value of interaction modeling in this context.

¹Accessible at snap.stanford.edu/jodie/#datasets.

	Wikipedia	Reddit	LastFM
Users	8,227	10,000	1,000
Items	1,000	1,000	1,000
Interactions	157,474	672,447	1,293,103
Unique edges	18,257	78,516	154,993

Table 3.1: Details of the datasets.

Table 3.1 summarizes the key characteristics of the three selected datasets. While Wikipedia and Reddit have comparable numbers of users and items, Reddit is significantly denser, with roughly four times more interactions. LastFM is denser still, with nearly 20 times the interaction density of Reddit. Notably, LastFM also exhibits a perfectly balanced user-to-item ratio, in contrast to the other two datasets.

3.2. Experimental framework

Evaluating embedding models is inherently complex due to the variety of input and output formats, as well as the diversity of training and inference procedures. Despite these differences, a fair and consistent evaluation across models must be ensured.

This complexity is further amplified in the case of temporal graphs, which can be interpreted in multiple ways depending on the structural and temporal aspects one wishes to emphasize. A temporal graph may be decomposed into a sequence of static snapshots taken at regular intervals, represented as a continuous time series of events, or treated as a dynamic structure where nodes and edges evolve over time [10]. These different interpretations offer varied trade-offs in terms of temporal resolution, scalability, expressiveness, and design opportunities, without any single approach being universally optimal.

The following subsections describe the design choices that guided the development of our evaluation framework². These are organized into four key components: data preparation, batching strategy, evaluation and training loop, and embedding comparison.

²<https://github.com/mazerti/link-prediction>

3.2.1. Data preparation

Each recorded interaction in the datasets provides three types of information: the identifiers of the interacting user and item, the timestamp of the interaction, and a set of optional features that offer additional context. In most cases, the evaluated models rely primarily on the user and item identifiers, along with the implicit temporal order of the interactions. As such, the framework consistently supplies user and item IDs in the exact sequence in which the interactions occur.

The framework also supports the inclusion of custom features in the input. These features can be specified either by the user through a configuration file or automatically requested by the model implementation during initialization. This flexibility ensures that models requiring specific features can be seamlessly integrated without manual intervention. Time-related features, in particular, benefit from this design. For example, some models use the time delta between successive interactions by the same user. While calculating this value at inference time would be computationally expensive, as it requires real-time tracking of each user’s previous interactions, it can be efficiently pre-computed when the full interaction history is available, reducing it to a straightforward query operation.

3.2.2. Batching Strategy

Temporal interaction modeling inherently involves a tradeoff between preserving the sequential nature of data and maximizing training efficiency through parallelism. As highlighted in prior work [19], [20], maintaining causality often comes at the cost of reduced parallelization capabilities. In the Jodie model, Kumar et al. addressed this by designing a graph-structure-aware batching strategy that retains temporal coherence while enabling some degree of parallel processing [19]. Meanwhile, Kefato et al. proposed an alternative in DeePRed by eliminating recursion entirely, replacing dynamic embeddings with static approximations to simplify training [20].

Drawing inspiration from the original LiMNet framework [11], we adopt a different approach: slicing the dataset into fixed-size sequences. The rationale is that sufficiently long sequences can serve as a reasonable approximation of the full interaction history. Each individual sequence is processed in temporal order, thereby preserving internal causal structure. However, because sequences are independent of one another, they can be processed in parallel, significantly accelerating training.

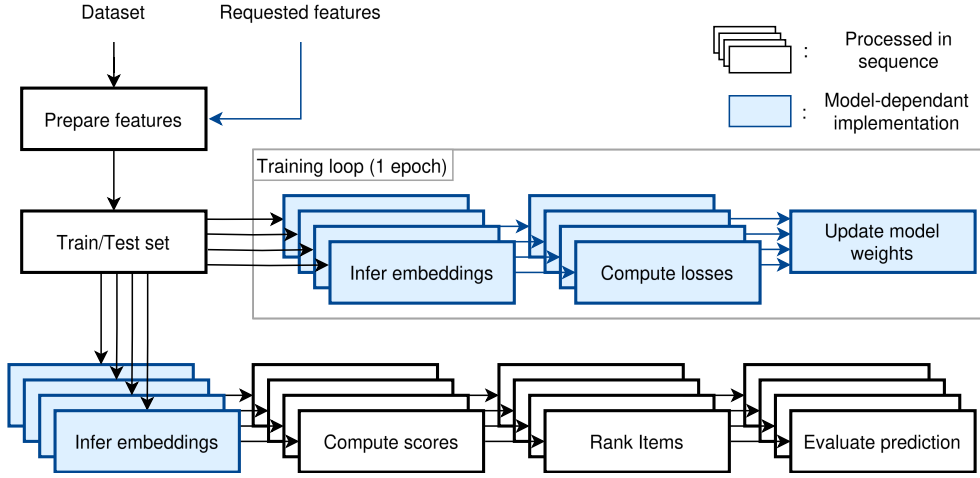


Figure 3.1: Schema of the evaluation framework. Blue indicates that the implementation is tight to the model evaluated.

This strategy offers a practical compromise. It enables the model to learn from temporally ordered data without incurring the full computational burden of processing the entire dataset sequentially. Moreover, it allows for flexibility in choosing the sequence length, which can be tuned to balance modeling capacity and computational efficiency. In our experiments, we found that shorter sequences often performed comparably well, suggesting that most relevant predictive signals are contained in recent interaction history.

3.2.3. Evaluation and Training loop

Designing a framework that accommodates any model for a given task is inherently challenging, as different models are often developed based on varying problem formulations. One major difference lies in the structure of the inputs. As discussed in Section 3.2.1, the framework addresses this by leveraging input features to bridge discrepancies between models. Another key distinction involves the nature of the outputs. Although all models ultimately aim to identify relevant items, they approach this goal in different ways. In the context of interaction prediction, predictions can be generated by computing embeddings to be compared, by directly scoring items, or by estimating the likelihood of future user-item interactions.

This framework solely addresses the model creating user and item embeddings, simplifying the evaluation process and allowing it to be performed independently of the model that generates them. However, all training and

loss evaluation logic is encapsulated within the model implementations. This approach allows for diverse optimization strategies, including loss functions that depend on a model’s internal memory state rather than exclusively on its outputs. These design choices are reflected in the framework architecture diagram presented in Figure 3.1.

3.2.4. Embedding comparison

The final challenge in the implementation concerns the use of embeddings. While embeddings are created to condense and represent interaction information, they are not the system’s ultimate objective. The actual goal is to rank items for a given user such that the item with which the user will interact appears as high as possible on the list.

There are several ways to translate embeddings into rankings. Since this is not the central focus of the current work, we adopt a straightforward approach: ranking item embeddings based on their proximity to the user embedding. To accommodate the diversity of models tested, the framework supports two proximity metrics.

The first is the dot product of normalized embeddings, which is equivalent to the cosine similarity between vectors:

$$\text{dot_product_score}(e^{\text{user}}, e^{\text{item}}) = \frac{e^{\text{user}}}{\|e^{\text{user}}\|} \cdot \frac{e^{\text{item}}}{\|e^{\text{item}}\|} \quad (5)$$

A higher dot-product score indicates that the item embedding is more closely aligned with the user embedding, and thus should be ranked higher.

The second metric is the L2 distance, a generalization of Euclidean distance to k -dimensional space:

$$\text{L2_score}(e^{\text{user}}, e^{\text{item}}) = \sqrt[k]{\|e^{\text{user}} - e^{\text{item}}\|^k} \quad (6)$$

For this score, smaller values correspond to closer embeddings and are therefore ranked higher.

In practice, we find that dot-product scoring yields slightly better predictions when applied to normalized models (as described later on, in Section 3.3.3), whereas L2 distance performs better for the more straightforward free-form embeddings such as those used by Jodie. To ensure a fair comparison across all models, performance in all experiments is measured using both scoring

methods, with only the best prediction results from either method being reported.

3.2.5. Code Reusability

In exploratory research projects like this one, writing the entire codebase from scratch can be advantageous. This approach eliminates the burden of dependency management, avoids the need to thoroughly understand legacy implementations, and frees the researcher from conforming to existing frameworks. Building from the ground up also allows for alternative perspectives on the task at hand and allows researchers to concentrate on challenges arising from novel aspects of the work. However, the ability to reproduce experiments and to reuse existing models, either as baselines or as foundations for further development, remains critically important, it would thus be inappropriate to write such a framework without having future researchers in mind.

To balance these needs, the framework has been designed with clarity, reproducibility, and extensibility in mind. Three principles have guided its development: comprehensive documentation, centralized state management, and a functional programming approach. Every function in the framework is systematically documented to help future researchers quickly grasp the implementation, whether to reuse the code or replicate its behavior in a new context.

To further streamline usability, state management has been centralized in a single component: the Context class. This class acts as a unified store for all stateful elements of the framework, ensuring that any part of the system can access necessary state variables with minimal effort.

In addition, the framework adheres to a functional programming-inspired style, favoring pure functions wherever possible for their conceptual simplicity and consistency. While this approach enhances readability and modularity, certain components, most notably the PyTorch modules, had to follow an object-oriented structure due to the requirements of external libraries.

This balance between clean design and practical flexibility ensures that the framework is easy to understand for future experiments.

3.3. Adaptations of the LiMNet architecture

The primary goal of this work is to evaluate the performance of the LiMNet model on the link prediction task. As discussed in Section 2.5, the original implementation of LiMNet includes input and output mapping layers, as well as a response layer specifically designed for IoT botnet detection. For our purposes, these components were removed to better align the model with the requirements of interaction prediction.

3.3.1. Loss functions

The loss function also required adaptation. Unlike botnet detection, link prediction is not a classification task, making the original cross-entropy loss unsuitable. Instead, we opted for a composite loss combining two components.

The first is an objective loss that minimizes the distance between the embeddings of interacting users and items. This can be computed either as the mean squared error between the embeddings of the interacting user and item or, when embeddings are normalized, as the squared difference between their dot product and 1 (more on this in Section 3.3.3).

However, the objective loss alone is insufficient to train the model effectively. Neural networks tend to converge to trivial solutions if not properly constrained; in this case, minimizing only the distance between embeddings would eventually cause all embeddings to collapse to the same value. To mitigate this, we introduce a regularization loss that promotes information retention by maximizing the distance between different users' embeddings and between different items' embeddings.

This regularization loss is computed as:

$$L_{\text{reg}} = \mathbf{U}\mathbf{U}^T + \mathbf{I}\mathbf{I}^T \quad (7)$$

where $\mathbf{U}$ and $\mathbf{I}$ are the matrices containing all user and item embeddings, respectively.

In addition to simplifying the architecture and adapting the loss function, we propose three modifications aimed at enhancing the model's performance: the addition of time features, embedding normalization, and the stacking of multiple LiMNet layers.

3.3.2. Addition of time features

While LiMNet leverages the order of interactions to propagate information in a causal manner, it does not incorporate actual timestamps when computing embeddings. We hypothesized that this omission could lead to a loss of valuable temporal information that might otherwise help in predicting the most relevant items. To test this assumption, we introduced time-based features aimed at capturing when each interaction occurred. Specifically, we sought to model cyclic behavioral patterns such as differences in user activity between weekdays and weekends, or between day and night.

Such patterns emerge in user-item interactions when certain types of items are more suited to specific activities. For example, work-related items are typically accessed more during office hours, whereas leisure-related items tend to see higher engagement in the evenings and on weekends. In the case of global services, these patterns may also reflect variations in popularity across different time zones. Figure 3.2 illustrates this phenomenon using two tracks from the LastFM dataset, each displaying distinct preferred listening times.

However, our datasets only include relative timestamps, which obscure the exact timing of interactions. As a workaround, we approximated cyclic patterns by applying a frequency decomposition to the timestamps. We computed two features to represent temporal cycles:

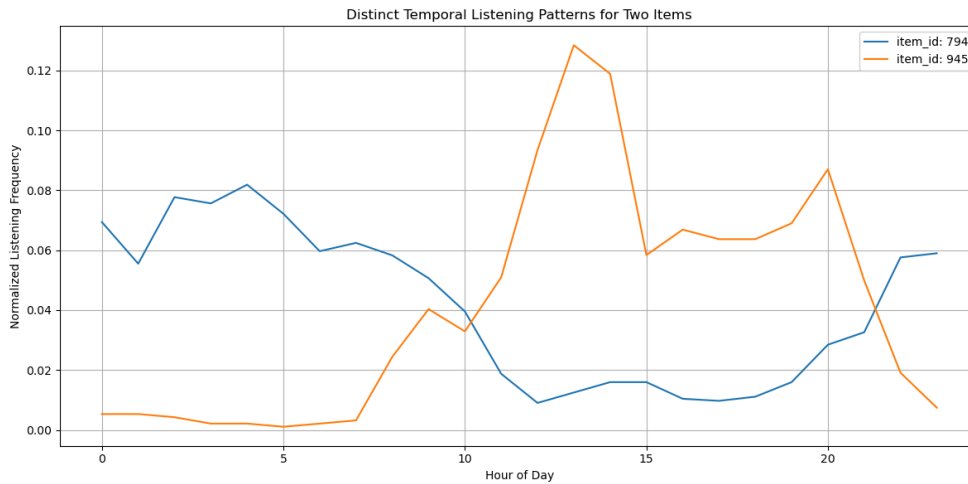


Figure 3.2: Example of two items with significantly distinct daily patterns within the LastFM Dataset.

$$\cos\left(\frac{2\pi t}{\Delta}\right), \sin\left(\frac{2\pi t}{\Delta}\right) \quad (8)$$

where t is the timestamp of the interaction, and Δ is the duration of the pattern to be captured (e.g., one day or one week) expressed in the timestamp’s time unit. This representation aims to provide the model with a more learnable form of temporal information, as machine learning models often struggle to extract patterns from raw one-dimensional values.

3.3.3. Embedding normalization

An efficient approach to computing dot-product scores (see Eq. 5) is to normalize all embeddings onto the unit sphere. In our implementation, we extended this normalization to include the embeddings stored in LiMNet’s memory. By ensuring that inputs to the cross-RNN mechanism are also normalized, we encourage the model to encode information primarily through angular relationships rather than magnitude.

This normalization has additional benefits. While the regularization loss is designed to prevent all embeddings from converging to the same direction, it does not explicitly prevent them from collapsing to zero. Since embeddings with a magnitude of zero are still orthogonal, they technically satisfy the regularization condition. Maintaining normalization throughout the network guards against this collapse by constraining all embeddings to lie on the surface of the unit sphere.

Our experiments showed a significant performance improvement with normalized embeddings. As a result, this modification was adopted as the default setting for all LiMNet -based experiments in this project.

3.3.4. Stacking of multiple LiMNet layers

The final adaptation we explored involved stacking multiple layers of the LiMNet architecture to form a deep recurrent neural network. This hierarchical design enhances the model’s representational capacity by allowing it to learn progressively more abstract features across layers. Figure 3.3 illustrates the structure of the extended model.

To introduce non-linearity and further boost the expressiveness of the network, leaky ReLU activation functions were inserted between each pair of layers. This design choice aims to help the model capture more complex patterns in user-item interactions.

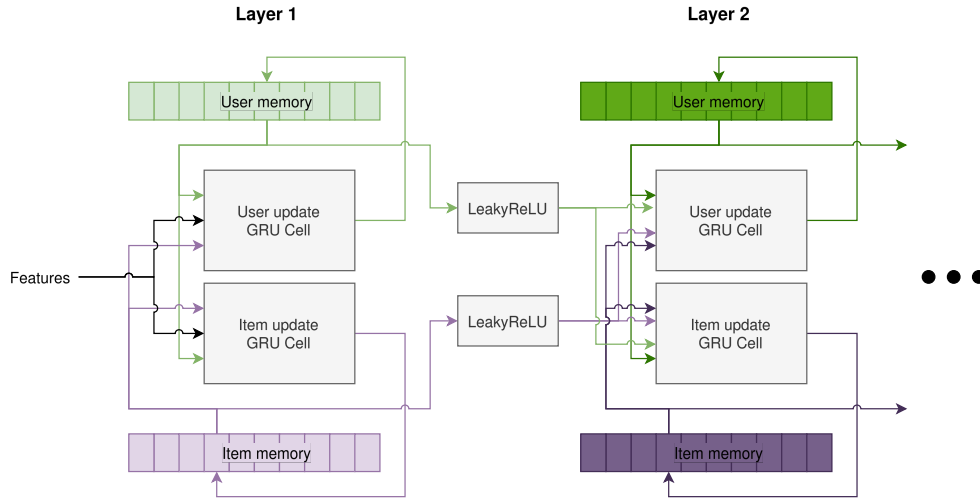


Figure 3.3: Architecture of LiMNet with two layers.

3.4. Baseline

We evaluated the performance of LiMNet against Jodie, a state-of-the-art cross-RNN model for learning embeddings in temporal interaction networks, described in further details in Section 2.6. We also attempted to implement DeePRed [20], but were unable to reproduce the performance reported in the original paper. As a result, DeePRed was excluded from our experimental comparisons.

Chapter 4

Experiments

This chapter presents the experiments conducted over the course of the project, along with their corresponding results. Each model was trained and evaluated across 7 to 10 runs, with different random seeds used to initialize weights in each run to ensure robustness. Performance was measured using Mean Reciprocal Rank (MRR), calculated based on the predicted ranks of the true target items in the test set. During evaluation, interactions were processed sequentially using windows of the same length as those employed during training.

Figure 4.1 presents a central finding of this thesis: a comprehensive performance comparison between LiMNet and Jodie across all datasets. The results unambiguously demonstrate that Jodie outperforms LiMNet by a considerable margin in every scenario. This outcome provides a strong, albeit negative, answer to the research question introduced at the beginning of this work. Further experiments were conducted to investigate whether LiMNet’s performance could be enhanced through architectural adjustments. These efforts are detailed in Section 4.1. Additionally, Section 4.2 explores a serendipitous insight that emerged during experimentation, offering a deeper understanding of the role temporal information plays in model performance.

4.1. Improvements on LiMNet

To reduce the performance disparity between LiMNet and Jodie, we experimented with three architectural modifications. Each of the following subsections describes one of these adaptations along with its experimental evaluation.

4.1.1. Adding time features

As described in Section 3.3.2, we augmented LiMNet with cyclic time features derived from interaction timestamps. This was done to investigate whether such information could enhance the model’s ability to capture temporal patterns.

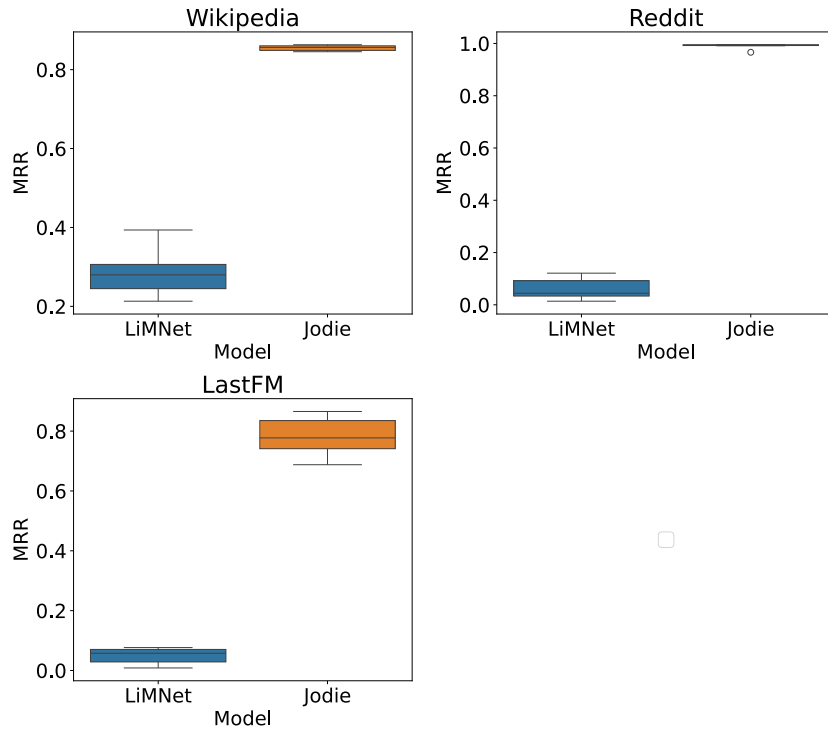


Figure 4.1: Comparison of LiMNet and Jodie models.

Figure 4.2 illustrates the impact of these features on the model’s performance, measured using MRR. While the additional features appear to reduce variance, they do not improve average performance and may even degrade it. These results suggest that LiMNet struggles to extract useful information from the added temporal signals, making this adaptation ineffective for improving its link prediction capability.

4.1.2. Normalizing the embeddings

As explained in Section 3.3.3, we proposed improving LiMNet’s performance by systematically normalizing the embeddings throughout the cross-RNN mechanism. This normalization encourages the model to learn angular relationships, which are more effectively exploited by the dot-product scoring function. It also prevents embeddings from collapsing to zero, which would diminish their representational power.

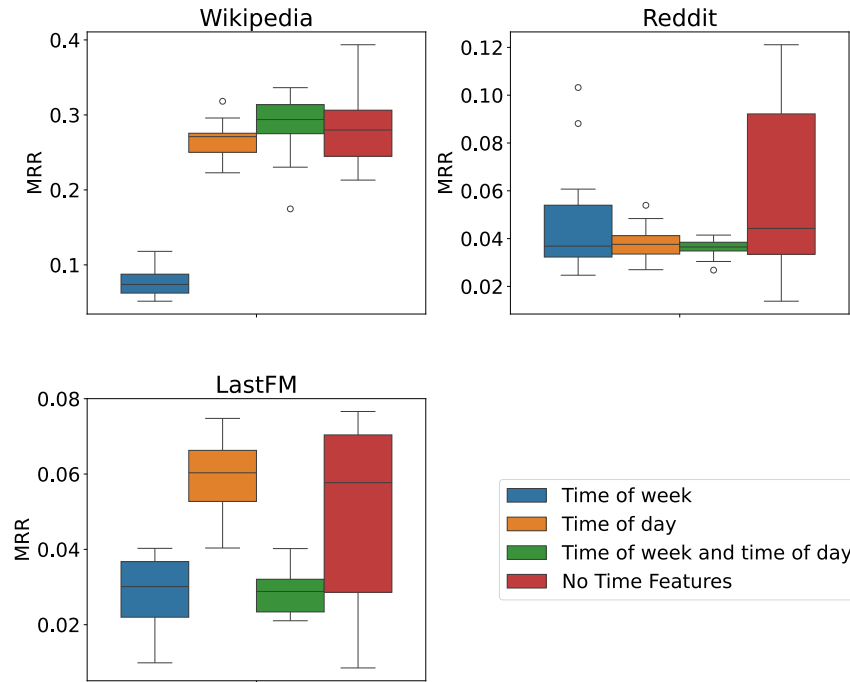


Figure 4.2: Performances of the LiMNet model with time features added.

As shown in Figure 4.3, this modification leads to a significant improvement in performance. Accordingly, we adopted embedding normalization as the default configuration for all other experiments with LiMNet .

4.1.3. Stacking layers

Following the enhancement outlined in Section 3.3.4, we investigated the impact of stacking multiple layers of the LiMNet architecture to determine whether increased depth could improve predictive accuracy by enhancing model expressiveness.

Figure 4.4 shows that stacking more than two layers does not result in further performance gains, and in some cases, leads to a decrease in accuracy. Interestingly, increasing from one to two layers improves performance on more complex datasets like Reddit and LastFM, but slightly degrades it on the simpler Wikipedia dataset. This suggests that additional layers may

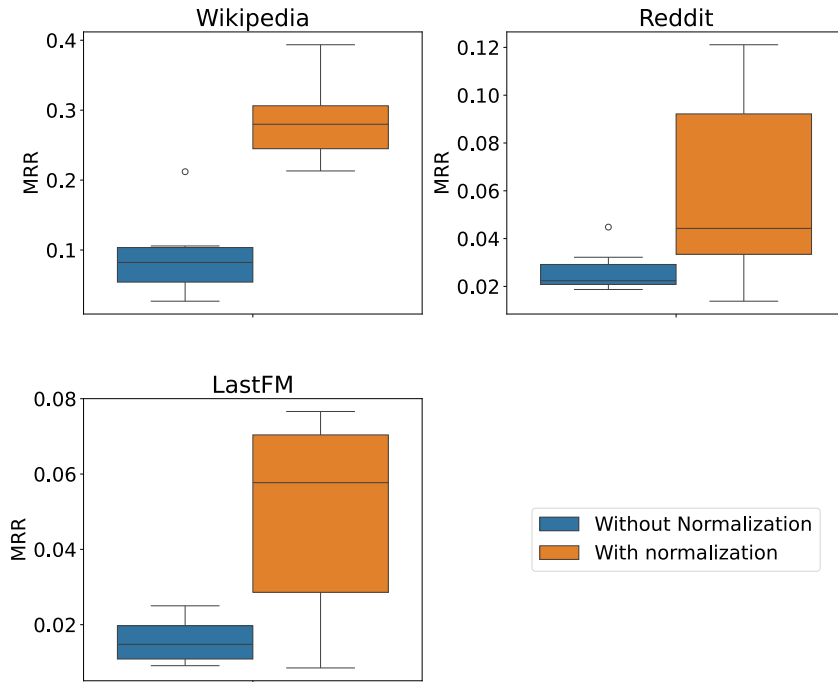


Figure 4.3: Performances of the LiMNet model with and without normalization of the embeddings.

help capture intricate interaction patterns, but are unnecessary, or even detrimental, in more straightforward settings.

4.2. Impact of the sequence size on the results

The final experiment examines how the length of the interaction sequence affects model performance, and by extension, the importance of temporal and sequential information. We trained and evaluated each model using sequence lengths of 16, 64, 256, and 1024, limiting how many past interactions were accessible for each prediction.

As illustrated in Figure 4.5, both models perform comparably across the tested sequence lengths, with longer sequences yielding only marginal gains or even slight degradations. The only notable exception is LiMNet on the Wikipedia dataset, where longer sequences show a modest improvement. These findings indicate that neither model significantly benefits from

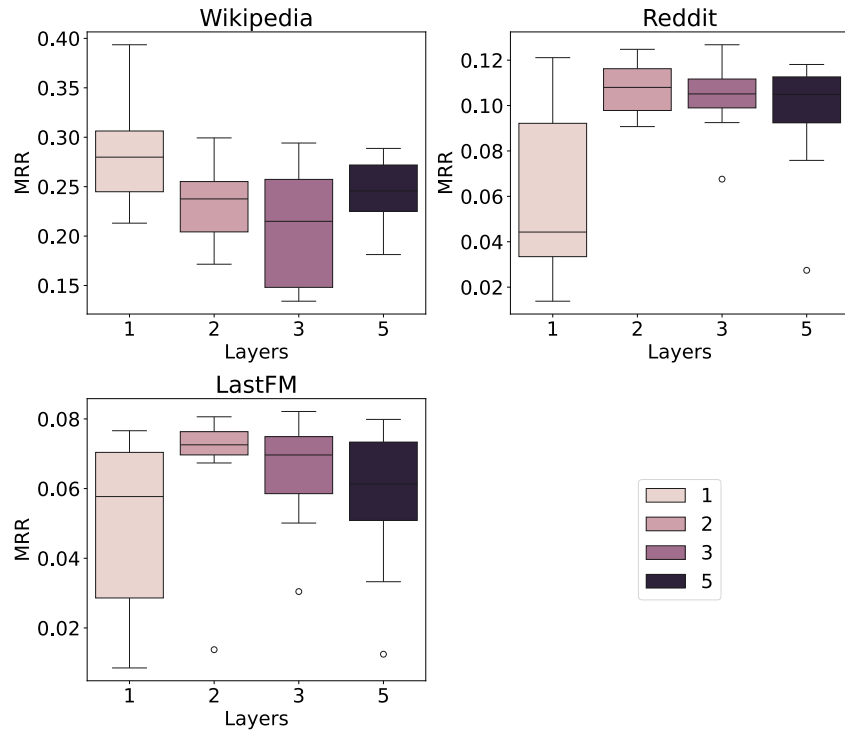


Figure 4.4: Performances of the LiMNet model with various numbers of stacked layers.

extended temporal context and may not effectively leverage long-term dependencies as originally intended.

Instead, these results suggest that both models are learning short-term global popularity trends rather than long-term, user-specific preferences. This behavior aligns with the characteristics of the datasets, where recent user activity can influence subsequent interactions. Furthermore, each dataset is limited to the 1,000 most popular items over a one-month period, likely favoring items with short-lived popularity spikes. As popularity shifts from one item to another, the models appear to focus on these transient dynamics.

This hypothesis may also explain why our implementation of Jodie achieves up to 60% higher MRR on the LastFM dataset compared to the original implementation [19]. A key difference lies in the batching strategy: our implementation processes interactions in strict chronological order, while

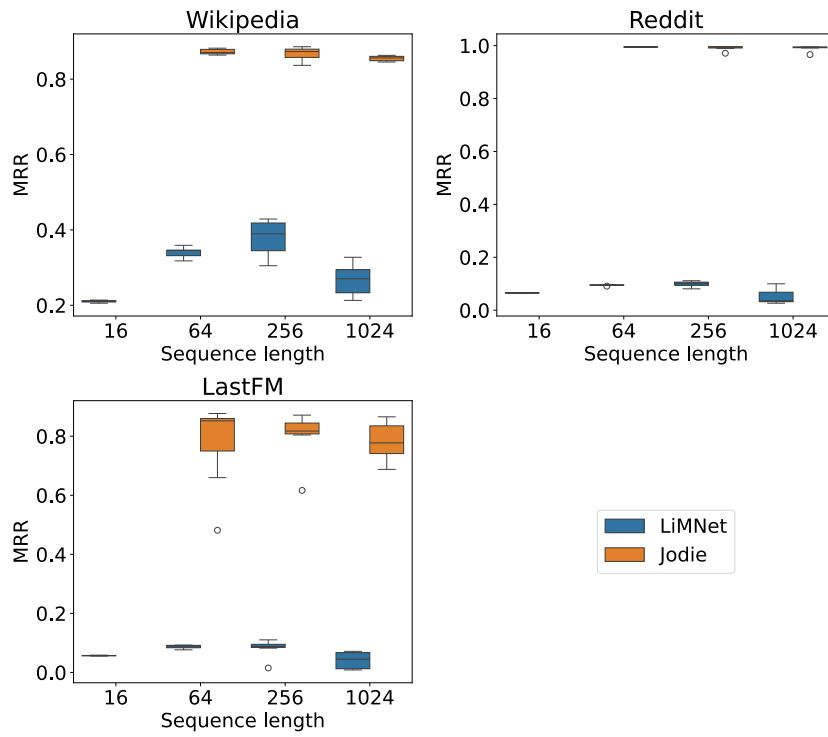


Figure 4.5: Effect of the sequence length on the models' performances.

the original uses the t-batch algorithm, which groups interactions by shared users or items. This distinction may result in our model better capturing global popularity signals, rather than local, user-specific patterns.

Chapter 5

Conclusions

The objective of this thesis was to evaluate the potential of the LiMNet model for interaction prediction in user-item dynamic networks. Addressing this research question led to the development of an evaluation framework tailored to this task, the adaptation and assessment of multiple LiMNet variants, and the reimplementing of the Jodie model as a state-of-the-art baseline.

This study yielded three key findings:

1. LiMNet significantly underperforms on the interaction prediction task when compared to Jodie.

Stripped down to its core cross-RNN memory module, LiMNet is clearly unable to match Jodie's performance across all datasets. Notably, the performance gap widens on more complex datasets, suggesting that Jodie's additional architectural components, such as its static embeddings and projection mechanism, play a critical role in enhancing predictive capability. It is important to emphasize that the version of LiMNet tested here excludes several components included in its original architecture. This simplification may explain why LiMNet performs poorly in this context, despite achieving strong results in domains like IoT botnet detection [11] and cryptocurrency fraud detection [12].

2. Embedding normalization within the cross-RNN mechanism leads to a substantial improvement in LiMNet's performance.

By maintaining normalized embeddings in memory, the model is encouraged to encode information through angular differences rather than magnitude. This alignment with the dot-product scoring method (i.e., cosine similarity) ensures that the memory state and the scoring function are consistent, which in turn improves the overall performance of the model.

3. Sequence length has minimal impact on model performance

The performance of both LiMNet and Jodie remained relatively stable when trained and evaluated on significantly shorter sequences. This stability implies that the models primarily learn from recent and globally popular interaction patterns (i.e. trends) rather than from long-term, user-specific histories. This behavior is likely influenced by the dataset construction, which emphasizes the most popular items and most active users over a fixed time period.

5.1. Limitations

To maintain a manageable scope, certain questions were left aside during the research process of this thesis. One of them is that the findings are specific to the task of user-item interaction prediction and do not generalize to broader link prediction scenarios. Additionally, the study focused primarily on the generation of embeddings, with less emphasis on how to most effectively leverage them for downstream tasks.

Performance evaluation at inference time was also not addressed, even though this is one of LiMNet’s key architectural advantages, as demonstrated in prior work [11]. This omission should be considered when interpreting the model’s potential and drawing conclusions about its practical utility.

5.2. Future Works

The sequence length experiment discussed in Section 4.2 revealed a novel and surprising insight into the mechanisms underlying user-item interactions. To validate and extend these observations, future work should test both LiMNet and Jodie on datasets that exhibit more complex, long-term, and localized behavioral patterns.

Another promising avenue for future research is exploring the significant performance gap between LiMNet and Jodie. Despite sharing a common core, the two models demonstrate dramatically different capabilities. An ablation study focusing on Jodie’s architectural components could shed light on which design decisions contribute most to its superior performance.

5.3. Ethics and Sustainability

5.3.1. Ethical aspects

As machine learning systems increasingly rely on large-scale data and compute resources, there is growing concern about widening inequalities that favor large institutions. Additional research on the LiMNet architecture could offer potential mitigation through three architectural and practical strengths:

1. Its scalability and lightweight design reduce infrastructure requirements, lowering the barrier to entry.
2. Ongoing efforts to develop decentralized and collaborative versions of LiMNet [22] could empower communities to deploy models using distributed resources.
3. This project's public availability enhances accessibility for regulators and policymakers, contributing to more informed legislative decision-making in an area still filled with uncertainty.

5.3.2. Societal impact

Better observed performances could have contributed to improve the performance and cost of retrieval pipelines, enabling people with better and faster access to information at a reduced cost. Unfortunately, with the underwhelming predictions performed by LiMNet compared to the state-of-the-art baseline, this project lost its impact potential and is therefore mostly dedicated to an academic use.

5.3.3. Sustainability

While LiMNet's efficiency had the potential to reduce the cost of computation, it is still unclear whether improved efficiency translates into net energy savings in the long term [23]. This consideration highlights not only the need for continued attention to the ecological impact of machine learning research and deployment, but more importantly the importance of societal awareness about sustainability compared with innovation.

References

- [1] M. G. Rodriguez, K. Gummadi, and B. Schoelkopf, “Quantifying Information Overload in Social Media and Its Impact on Social Contagions,” *Proceedings of the International AAAI Conference on Web and Social Media*, vol. 8, no. 1, pp. 170–179, May 2014, doi: [10.1609/icwsm.v8i1.14549](https://doi.org/10.1609/icwsm.v8i1.14549).
- [2] J. Batista and P. Marques, “An Overview on Information and Communication Overload,” *Information and Communication Overload in the Digital Age*, pp. 1–20, 2017. doi: [10.4018/978-1-5225-2061-0.ch001](https://doi.org/10.4018/978-1-5225-2061-0.ch001).
- [3] L. Shahrzadi, A. Mansouri, M. Alavi, and A. Shabani, “Causes, Consequences, and Strategies to Deal with Information Overload: A Scoping Review,” *International Journal of Information Management Data Insights*, vol. 4, no. 2, p. 100261, Nov. 2024, doi: [10.1016/j.jjime.2024.100261](https://doi.org/10.1016/j.jjime.2024.100261).
- [4] B. P. Knijnenburg, M. C. Willemsen, Z. Gantner, H. Soncu, and C. Newell, “Explaining the User Experience of Recommender Systems,” *User Modeling and User-Adapted Interaction*, vol. 22, no. 4, pp. 441–504, Oct. 2012, doi: [10.1007/s11257-011-9118-4](https://doi.org/10.1007/s11257-011-9118-4).
- [5] J. Teevan, S. T. Dumais, and E. Horvitz, “Beyond the commons: Investigating the value of personalizing web search,” in *PIA 2005 - Workshop on New Technologies for Personalized Information Access, Proceedings of the 1st International Workshop - Held in Conjunction with: 10th Int. Conference on User Modeling, UM 2005*, 2005, pp. 84–92.
- [6] D. W. Oard and J. Kim, “Implicit Feedback for Recommender Systems.”
- [7] Y. Hu, Y. Koren, and C. Volinsky, “Collaborative Filtering for Implicit Feedback Datasets,” in *2008 Eighth IEEE International Conference on Data Mining*, Dec. 2008, pp. 263–272. doi: [10.1109/ICDM.2008.22](https://doi.org/10.1109/ICDM.2008.22).
- [8] S. Gheewala, S. Xu, and S. Yeom, “In-Depth Survey: Deep Learning in Recommender Systems—Exploring Prediction and Ranking Models, Datasets, Feature Analysis, and Emerging Trends,” *Neural Computing and Applications*, vol. 37, no. 17, pp. 10875–10947, Jun. 2025, doi: [10.1007/s00521-024-10866-z](https://doi.org/10.1007/s00521-024-10866-z).
- [9] V. Bogina, T. Kuflik, D. Jannach, M. Bielikova, M. Kompan, and C. Trattner, “Considering Temporal Aspects in Recommender Systems: A

- Survey,” *User Modeling and User-Adapted Interaction*, vol. 33, no. 1, pp. 81–119, Jul. 2022, doi: [10.1007/s11257-022-09335-w](https://doi.org/10.1007/s11257-022-09335-w).
- [10] Y. Zheng, L. Yi, and Z. Wei, “A survey of dynamic graph neural networks,” *Frontiers of Computer Science*, vol. 19, no. 6, p. 196323, Dec. 2024, doi: [10.1007/s11704-024-3853-2](https://doi.org/10.1007/s11704-024-3853-2).
- [11] L. Giaretta, A. Lekssays, B. Carminati, E. Ferrari, and S. Girdzijauskas, “LiMNet : Early-Stage Detection of IoT Botnets with Lightweight Memory Networks,” in *Computer Security – ESORICS 2021 : 26th European Symposium on Research in Computer Security, Darmstadt, Germany, October 4–8, 2021, Proceedings, Part I*, in Lecture Notes in Computer Science. Springer Nature, 2021. doi: [10.1007/978-3-030-88418-5_29](https://doi.org/10.1007/978-3-030-88418-5_29).
- [12] Z. Cui, “Classification of Financial Transactions using Lightweight Memory Networks,” 2022. [Online]. Available: <https://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-321923>
- [13] Y. Qu *et al.*, “Product-Based Neural Networks for User Response Prediction,” in *2016 IEEE 16th International Conference on Data Mining (ICDM)*, Dec. 2016, pp. 1149–1154. doi: [10.1109/ICDM.2016.0151](https://doi.org/10.1109/ICDM.2016.0151).
- [14] M. A. Hasan and M. J. Zaki, “A Survey of Link Prediction in Social Networks,” *Social Network Data Analytics*. Springer US, Boston, MA, pp. 243–275, 2011. doi: [10.1007/978-1-4419-8462-3_9](https://doi.org/10.1007/978-1-4419-8462-3_9).
- [15] R. N. Lichtenwalter, J. T. Lussier, and N. V. Chawla, “New perspectives and methods in link prediction,” in *Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, in KDD '10. Washington, DC, USA: Association for Computing Machinery, 2010, pp. 243–252. doi: [10.1145/1835804.1835837](https://doi.org/10.1145/1835804.1835837).
- [16] B. Sarwar, G. Karypis, J. Konstan, and J. Riedl, “Item-Based Collaborative Filtering Recommendation Algorithms,” in *Proceedings of the 10th International Conference on World Wide Web*, Hong Kong Hong Kong: ACM, Apr. 2001, pp. 285–295. doi: [10.1145/371920.372071](https://doi.org/10.1145/371920.372071).
- [17] P. Covington, J. Adams, and E. Sargin, “Deep Neural Networks for YouTube Recommendations,” in *Proceedings of the 10th ACM Conference on Recommender Systems*, in RecSys '16. New York, NY, USA: Association for Computing Machinery, Sep. 2016, pp. 191–198. doi: [10.1145/2959100.2959190](https://doi.org/10.1145/2959100.2959190).

- [18] H. Dai, Y. Wang, R. Trivedi, and L. Song, “Deep Coevolutionary Network: Embedding User and Item Features for Recommendation,” no. arXiv:1609.03675, arXiv, Feb. 2017. doi: [10.48550/arXiv.1609.03675](https://doi.org/10.48550/arXiv.1609.03675).
- [19] S. Kumar, X. Zhang, and J. Leskovec, “Predicting Dynamic Embedding Trajectory in Temporal Interaction Networks,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, in KDD '19. Anchorage, AK, USA: Association for Computing Machinery, 2019, pp. 1269–1278. doi: [10.1145/3292500.3330895](https://doi.org/10.1145/3292500.3330895).
- [20] Z. Kefato, S. Girdzijauskas, N. Sheikh, and A. Montresor, “Dynamic Embeddings for Interaction Prediction,” in *Proceedings of the Web Conference 2021*, in WWW '21. Ljubljana, Slovenia: Association for Computing Machinery, 2021, pp. 1609–1618. doi: [10.1145/3442381.3450020](https://doi.org/10.1145/3442381.3450020).
- [21] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, “Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling,” 2014. [Online]. Available: <https://arxiv.org/abs/1412.3555>
- [22] L. Giaretta, A. Lekssays, B. Carminati, E. Ferrari, and S. Girdzijauskas, “Metasoma: Decentralized and Collaborative Early-Stage Detection of IoT Botnets.”
- [23] K. Gillingham, D. Rapson, and G. Wagner, “The Rebound Effect and Energy Efficiency Policy,” *Review of Environmental Economics and Policy*, vol. 10, no. 1, pp. 68–88, 2016, doi: [10.1093/reep/rev017](https://doi.org/10.1093/reep/rev017).

Appendix A

Complexity

In this section we perform a formal complexity analysis of both LiMNet and Jodie regarding memory usage at inference. Training is left aside as it can be tweaked through the usage of varied batching algorithms, but also because the cross-RNN mechanisms studied in this thesis are designed for performance at inference with little consideration for the training time.

Table 1.1 lists the symbols used in this complexity analysis.

The memory usage of both models are divided into two part: the embedding memory that stores the dynamic embeddings of every user and item in the network, and the model's weights.

The embedding memory is the same for LiMNet and Jodie, it stores for each user and item an embedding of size d . Therefore, the memory requirements for it is $O(d(u + i))$.

The model weights vary between LiMNet and Jodie, yet both models share the same cross-RNN mechanism that contains a RNN/GRU cell with inputs of size $O(d + f)$ and outputs of size d , for a total memory complexity of $O(d^2 + df)$.

Symbol	Meaning	Context
u	Number of users in the network	Dynamic value that can scales from thousands to millions depending on the success of the application
i	Number of items in the network	Dynamic value that can scales from thousands to millions depending on the application
d	Size of the embeddings	Fixed value, typically small
f	Size of the input features	Fixed value, typically small

Table 1.1: Symbols used for the complexity analysis.

	LiMNet	Jodie
Embedding memory	$O(d(u + i))$	$O(d(u + i))$
Model weights	$O(d^2 + df)$	$O(d^2 + d(f + u + i) + ui)$
Total complexity	$O(d^2 + d(u + i + f))$	$O(d^2 + d(f + u + i) + ui)$
Assuming $d, f \ll u, i$	$O(u + i)$	$O(ui)$

Table 1.2: Summary of the memory complexity for LiMNet and Jodie

In addition, Jodie have three outputs modules to take into account: the temporal projection module, the one-hot encoding module and the prediction layer. The temporal projection layer is a linear layer with input of size 1 (the time delta), the complexity of this module is $O(d)$. The one hot encoding module is essentially memory-free. However, it increase the size of the embeddings fed into the prediction layer and served as output of the models. Therefore, the prediction layer that is also a linear layer, have a complexity of $O((d + u)(d + i)) = O(d^2 + d(u + i) + ui)$.

Table 1.2 presents a summary of the total complexity for both models. It stands out that the combination of the one-hot encoding module and the prediction layer causes Jodie to be an order of magnitude more resource intensive at inference than LiMNet.

TRITA – EECS-EX 2024:0000

Stockholm, Sweden 2025

www.kth.se